

## ITI121 Assignment 2

### Introduction

In this assignment, you will apply the knowledge and skills that you learned about Object Detection to train your own object detector for custom objects.

### Tasks

This is an **INDIVIDUAL** assignment.

For this assignment, you are expected to fine-tune a pretrained Object Detection model to detect **2 classes** of objects you selected. Both object classes **SHOULD NOT** be from COCO object class (refers to [https://github.com/amikelive/coco-labels/blob/master/coco-labels-2014\\_2017.txt](https://github.com/amikelive/coco-labels/blob/master/coco-labels-2014_2017.txt)).

The following are the tasks you will need to complete:

- Collect and annotate the images using either Roboflow or LabelStudio.
- Train the model and evaluate the model performance based on mean average precision.
- Log your experiments using tools like Weights & Biases (wandb).
- Improve your model through fine-tuning your dataset, hyper-parameter, data augmentation, and any other techniques and compare the model performances.
- Export your model to a most performant format for a chosen platform (e.g. CPU or GPU) capable of performing fast inference on ONE selected test image and one test video (around 1 minute duration).
- Deploy the model to Hugging Face (refers to Appendix for instructions)
- Create a Gradio demo app and deploy the app to Hugging Face to demo your model (refers to Appendix for instructions)

### Submission

You are to submit the following, in a single zip file, to POLITEmail:

1. A report (maximum 5 pages, font size 12, single column) that contains the following:
  - discussion of your data collection and annotation process and include the link to your dataset/annotation.
  - The finetuning process to improve your model performance and comparison of model performance.

- screenshots of your experimental log on weights and biases
  - link to Hugging Face model repo and space
2. ONE test image (original and predicted) and ONE test video file (original and predicted)

**Total Marks: 35 marks**

| Criteria                           | Inadequate                                                                                                              | Acceptable                                                                                | Satisfactory                                                                                                            | Good                                                                                                                                                  | Excellent                                                                                                                                                |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data Collection</b><br>10 marks | Very limited data collection (e.g. <20 images per class or only one class). Poor or missing annotations.<br><br>0 to <2 | Minimal dataset collected; some bounding boxes misaligned or inconsistent.<br><br>2 to <4 | Sufficient dataset (>20 images/class) with mostly correct annotations; somewhat relevant to task.<br><br>4 to <6        | Good dataset (>40 images/class); well-annotated and relevant to task.<br><br>6 to <8                                                                  | Extensive dataset (>60 images/class); highly relevant, diverse, and accurately annotated.<br><br>8 to 10                                                 |
| <b>Model Training</b><br>15 marks  | Trained with minimal effort; poor performance; no fine-tuning or improvements.<br><br>0 to <3                           | Some training attempted but weak results; little evidence of tuning.<br><br>3 to <6       | Reasonable training with one fine-tuned model; average precision (measured by <a href="#">mAP@0.5</a> ).<br><br>6 to <9 | Multiple fine-tuning attempts with parameter tuning and improvements; above-average precision. (measured by <a href="#">mAP@0.5</a> ).<br><br>9 to 12 | Extensive experimentation and optimization; model achieves high precision and clear improvement. (measured by <a href="#">mAP@0.5</a> ).<br><br>12 to 15 |
| <b>Documentation</b><br>5 marks    | Unclear or incoherent writing; missing key details.<br><br>0 to <1                                                      | Basic report with limited discussion of results.<br><br>1 to <2                           | Reasonably clear writing; adequate discussion of methods and results.<br><br>2 to <3                                    | Well-written, coherent report; good analysis of results.<br><br>3 to <4                                                                               | Excellent clarity and depth; insightful discussion with clear evidence of understanding.<br><br>4 to 5                                                   |

|                                  |                                                                                    |                                                                                                                  |                                                                                                        |                                                                                                     |                                                                                                       |
|----------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Deployment</b><br><br>5 marks | Model not deployed or fails to run.<br><br><br><br><br><br><br><br><br><br>0 to <1 | Model deployed but with major errors or incomplete functionality.<br><br><br><br><br><br><br><br><br><br>1 to <2 | Model deployed successfully but with limited usability.<br><br><br><br><br><br><br><br><br><br>2 to <3 | Model runs well with minor UI or performance issues.<br><br><br><br><br><br><br><br><br><br>3 to <4 | Fully functional, responsive deployment with smooth UX.<br><br><br><br><br><br><br><br><br><br>4 to 5 |
|----------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|

## APPENDIX

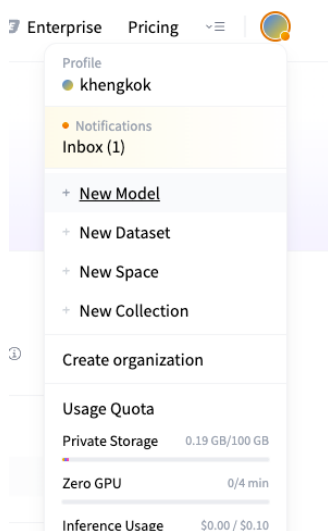
### Preliminary

1. Create an account at Hugging Face: <https://huggingface.co/>
2. Join the organization **ITI121-2025S2** using this link:

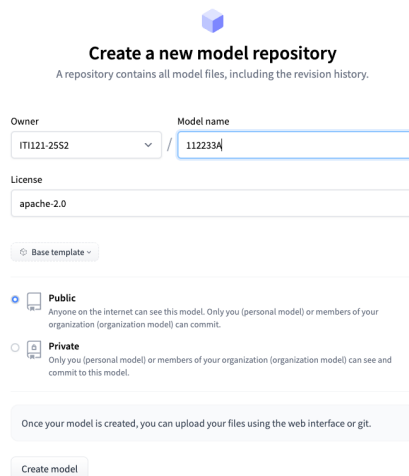
<https://huggingface.co/organizations/ITI121-25S2/share/oHOsfxPpWHFbKNExAUYeWIOeeRpvWXBjsu>

### Hosting your model on HuggingFace

3. Click on your profile, and choose New Model:



4. In Create a new model repository, choose “ITI121-2025S2” as Owner and use your student number for model name (e.g., 112233A), use Apache 2.0 License. Choose Public for access, and then click Create model to create the repository.



**Create a new model repository**  
 A repository contains all model files, including the revision history.

Owner: ITI121-25S2 / Model name: 112233A

License: apache-2.0

Base template: [dropdown]

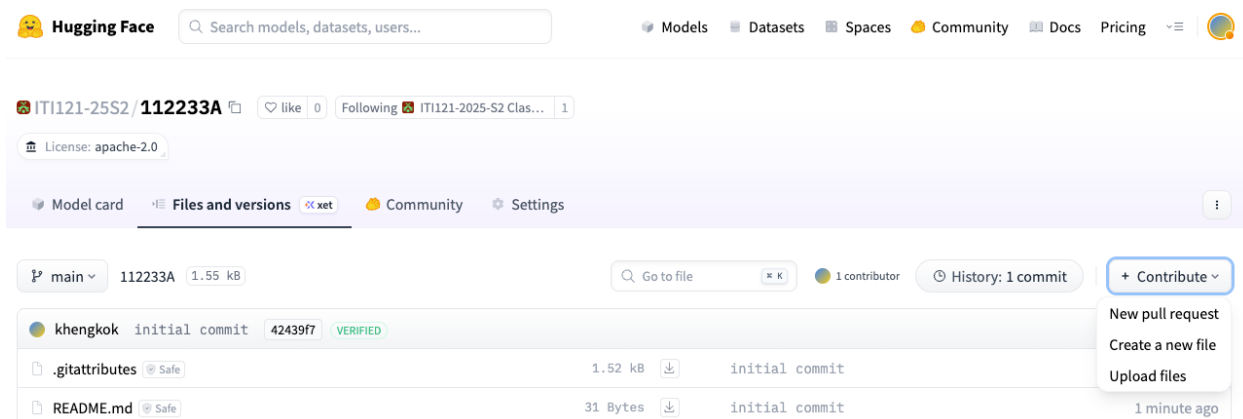
☒ **Public**  
 Anyone on the internet can see this model. Only you (personal model) or members of your organization (organization model) can commit.

☐ **Private**  
 Only you (personal model) or members of your organization (organization model) can see and commit to this model.

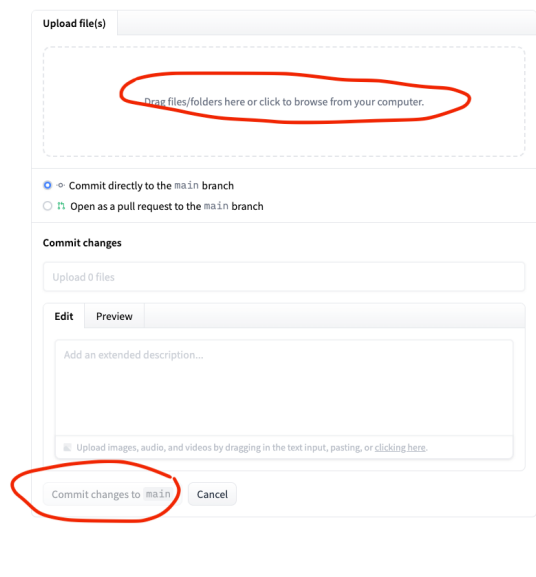
Once your model is created, you can upload your files using the web interface or git.

Create model

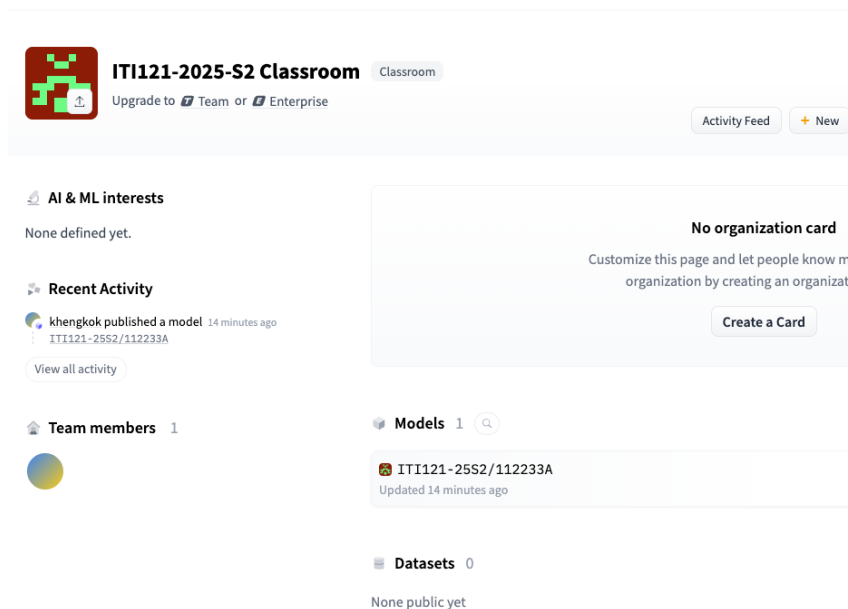
- In the model repository page, click on *Files and versions tab*. In the Files and versions, click on *+Contribute* and in the pull-down menu, click *Upload files* to upload your model (e.g. goldfish\_v2.pt) file.



- Click on the *Drag files/folders here or click to browse from your computer* and choose your model file (e.g. goldfish\_v2.pt). After that click on *Commit changes to main* to upload and commit your model file to the repo.



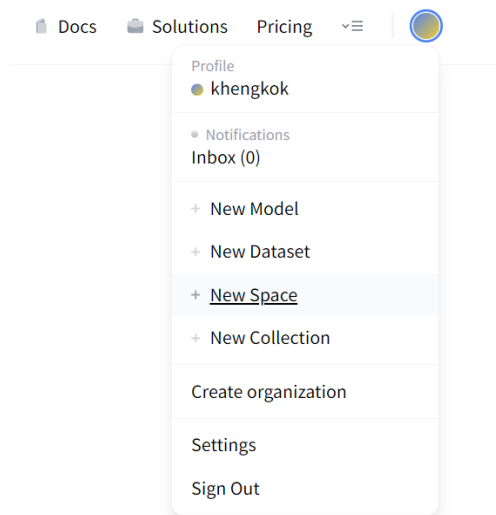
- In the ITI121-2025-S2 Classroom, you should be able to see your model:



Your model repo\_id (which you will need it later for your Streamlit/Gradio code) will be something like *ITI121-25S2/112233A*.

## Hosting your application on HuggingFace

8. Click on your profile, and choose New Space:



9. In "Create New Space" page, choose "ITI121-2025S2" as Owner and use your student number for space name (e.g., 112233A), use Apache 2.0 License, and choose Gradio for Space SDK. Select Public for the space visibility. Click "Create Space".

## Create a new Space

Spaces are Git repositories that host application code for Machine Learning demos.  
 You can build Spaces with Python libraries like [Gradio](#), or using [Docker images](#).

Owner: ITI121-25S2 / Space name: 112233A

Short description: Short Description

License: apache-2.0

Select the Space SDK  
 You can choose between [Gradio](#), [Docker](#), or [Static](#) to host your Space.


**Gradio**  
 4 templates


**Docker**  
 17 templates


**Static**  
 6 templates

Choose a Gradio template:

Blank

chatbot

text-to-image

leaderboard

Trackio

Space hardware: Free

CPU Basic

ZeroGPU

Custom Hardware

- Follow the instructions to clone the git repository to your local PC.

```

HTTPS SSH
# When prompted for a password, use an access token with write permissions.
# Generate one from your settings: https://huggingface.co/settings/tokens
git clone https://huggingface.co/spaces/ITI121-25S2/112233A
  
```

- Create your Gradio app to demo your model in the repository and then commit and push.

You can modify the codes from the following demo app:

[https://huggingface.co/spaces/khengkok/balloon\\_od/tree/main](https://huggingface.co/spaces/khengkok/balloon_od/tree/main)

Modify the codes, requirements file, README.md file accordingly. For the README file, change the title to your student number, e.g., 112233A.



*NOTE: Since Oct 1, 2025, hugging face no longer supports password authentication.*

*You need to update your git repo using the following commands:*

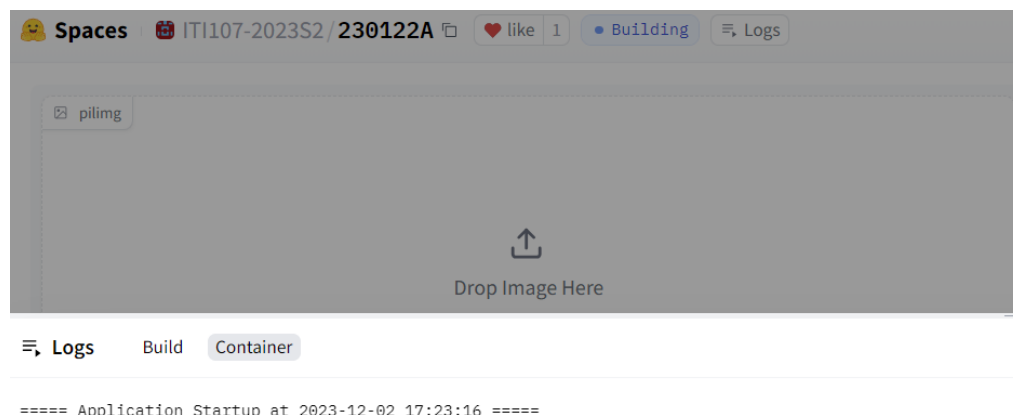
```
$: git remote set-url origin https://<user_name>:<token>@huggingface.co/<repo_path>
```

```
$: git pull origin
```

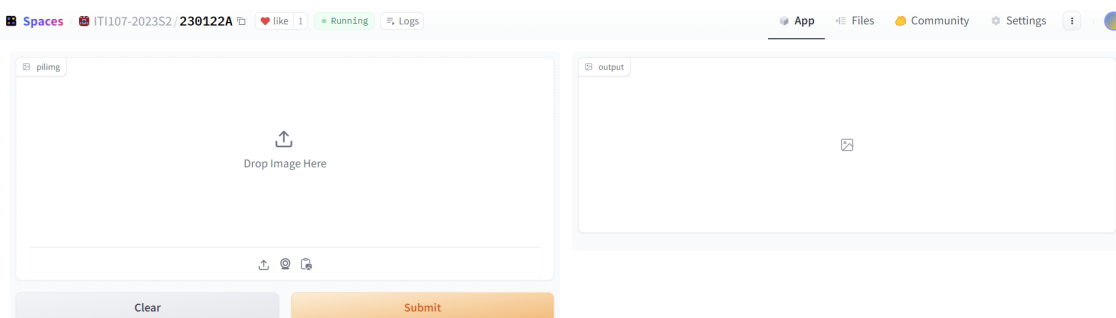
*e.g. if your access token is aa22bb33cc, and your repo is iti121-25s2/112233A, then type:*

```
git remote set-url origin https://khengkok: aa22bb33cc@huggingfae.co/spaces/iti121-25s2/112233A
```

After pushing, the build process on Hugging Face will start, resulting in a docker container running your application. You can click on the space to view your build process.



12. Once the container starts up, you can now access your Gradio demo app:



13. If there are any changes to the code, just push the changes to repo and this will rebuild your app on Hugging Face space.

## Create Access Token

14. Go to your profile page, under access token, create a token with write access

< **Create new Access Token**

Token type

Fine-grained Read **Write**

● This cannot be changed after token creation.

Token name

iti121-token

This token has read and write access to all your and your orgs resources and can make calls

Create token